CX651 — Assignment 8

JVM Observation Lab: Bytecode and Garbage Collection

Overview

In this assignment, you will look inside the JVM — not by writing `new` code, but by examining what the compiler produces. You will compile a provided class, read its bytecode using `javap`, trace a method call through the operand stack, and explain what each instruction does. You will then run a second program that stresses the heap, observe the garbage-collection log, and explain what the log reveals about how the JVM manages memory.

General Instructions

You will run several experiments and record your answers directly in this document, then submit it to Gradescope. The following files are provided:

- `IntMath.java` — an arithmetic class with three `static` methods. Compile it and examine its bytecode.
- `HeapStress.java` — a program that allocates many short-lived objects to trigger garbage collection. Run it with GC logging enabled and interpret the output.

Setup

Verify Java 21 or later is installed:

```
java -version   # must print 21 or higher

javac -version
```

Run all commands below from the directory containing the `.java` files.

Part A — Bytecode Reading

Step 1: Compile.

```
javac IntMath.java
```

Step 2: Disassemble.

```
javap -c IntMath
```

https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-6.html Keep the bytecode output visible as you answer the questions. Refer to the JVM Instruction Reference at as needed.

The `add` method

Q1. What is the first instruction in `add`? Where does its operand come from, and what does it push onto the stack?

iload_0. It loads the first integer argument from local slot 0 and pushes the value into the operand stack.

Q2. Trace the operand stack step by step through the entire `add` method. Write each step as: `[ ] → <instruction> → [stack contents, top on right]`.

[] → iload_0 → [a]
[a] → iload_1 → [a, b]
[a, b] → iadd → [a+b]
[a+b] → ireturn → []

Q3. What does `ireturn` do? Could the compiler use `lreturn` or `dreturn` here instead? Explain why or why not.

ireturn pops the integer value from the operand stack and returns it to the calling method. The compiler cannot use the lreturn or dreturn because the method's return type is integer(int). 1return is of type long and dreturn is of type double. Using either would not be compatible with the method's return type.

The `factorial` method

Q4. Find the instruction that performs the recursive call `factorial(n-1)`. Write its mnemonic. What does the mnemonic tell you about how the JVM handles dispatch for `static` methods?

invokestatic #7 // Method factorial:(I)I
invokestatic provides a static method without a receiver argument object. It does not do virtual dispatch based on the object's runtime class.

Q5. Identify the branch instruction in the `factorial` method and its target offset. Under what condition does the JVM take the branch? What happens if it does not?

2: if_icmpgt 7

Its target offset is 7. JVM takes the branch when n>1 for the given integers on the operand stack of n and 1. JVM does not take the branch if n<1 and falls to iconst_1 at offset 5 and ireturn at offset 6 to return 1.

Q6. Count the `invokestatic` instructions in the `factorial` method. Examine the method's control flow. Did `javac` preserve the recursive call or convert the method into an iterative loop? Support your answer using the bytecode. What operation occurs immediately after the recursive call returns?

11: invokestatic #7 // Method factorial:(I)I

javac preserves the recursive call. Each recursive call executes the same instruction and creates a new method call. After the recursive call is returned, the imul instruction at offset 14 multiplies the result by the original value of n.

Full annotation of the `square` method

Q7. Copy the complete bytecode for the `square` method and annotate every instruction. For each instruction provide: (a) its full name, (b) what it pops, (c) what it pushes. Use inline comments. Example format:

Annotation format:

```
<offset>: <instruction>  // Full name:
                         // Pops:
                         // Pushes:
```

Paste your annotated `square` bytecode here:

```
0: iload_0
// Full name: Load int from local var 0
// Pops: N/A
// Pushes: The integer argument x.
// Stack: [] → [x]

1: iload_0
// Full name: Load int from local var 0
// Pops: N/A
// Pushes: The integer argument x again
// Stack: [x] → [x, x]

2: imul
// Full name: Integer multiplication
// Pops: Two integers, x and x
// Pushes: Their product, x * x.
// Stack: [x, x] → [x * x]

3: ireturn
// Full name: Return int from method
// Pops: The integer result, x * x
// Pushes: N/A
// Return: x * x to the caller
```

Part B — Garbage-Collection Log Interpretation

Step 1: Compile `HeapStress.java`.

```
javac HeapStress.java
```

Step 2: Run with GC logging. The `-Xms` and `-Xmx` flags constrain the heap to make GC events easier to observe.

```
java -Xms16m -Xmx64m -Xlog:gc*:file=gc.log:time,uptime,level,tags HeapStress
```

Step 3: Open `gc.log` in a text editor. Examine the JVM initialization lines for Q8–Q9, then locate the garbage-collection events for the remaining questions. The first event is typically identified by `GC(0)`.

Q8. Find the JVM version/build information in the initialization section of `gc.log` and copy the complete line. Its wording may vary. For example, it may begin with `Version:`. If your log does not include this information, paste the output of `java -version` instead. Include vendor information if it appears; you do not need to obtain it separately.

Note: Exact wording, event types, and numerical values may vary by platform, garbage collector, and JVM version. Base your answers on your own output rather than matching the examples exactly.

```
[2026-10-23T22:44:41.365-0400][0.005s][info][gc,init] Version:
21.0.12+8-LTS (release)
```

Q9. Identify the garbage collector used in your run and copy the log line that identifies it. Give the collector's full name if the log uses an abbreviation. For example, `Using G1` identifies the Garbage-First (G1) collector.

```
[2026-10-23T22:44:41.364-0400][0.005s][info][gc     ] Using G1
```

Q10. Find the first garbage-collection event in `gc.log`. An event may span several lines; select its summary line, which shows heap usage before and after collection, heap capacity, and pause duration. Begin your answer with the exact uptime shown on that summary line, then copy the complete line. Identify: (a) heap in use before collection, (b) heap in use after collection, (c) total heap capacity at that event, and (d) pause duration.

```
[2026-10-23T22:44:41.376-0400][0.017s][info][gc          ]
GC(0) Pause Young (Normal) (G1 Evacuation Pause) 9M->1M(18M)
0.293ms
```

(a) Heap in use before collection: 9MB
(b) Heap in use after collection: 1MB
(c) Heap capacity at that event: 18MB
(d) Pause Duration: 0.293ms

Q11. Examine the first young-generation garbage-collection event. Describe the changes in Eden, survivor, and old-region counts, citing specific log lines. Explain what these changes suggest about reclaimed and surviving objects, and what they cannot tell you about individual objects.

```
[2026-10-23T22:44:41.376-0400][0.017s][info][gc,heap     ]
GC(0) Eden regions: 8->0(8)
[2026-10-23T22:44:41.376-0400][0.017s][info][gc,heap     ]
GC(0) Survivor regions: 0->1(1)
[2026-10-23T22:44:41.376-0400][0.017s][info][gc,heap     ]
GC(0) Old regions: 2->2
[2026-10-23T22:44:41.376-0400][0.017s][info][gc,heap     ]
GC(0) Humongous regions: 0->0
[2026-10-23T22:44:41.376-0400][0.017s][info][gc,metaspace]
GC(0) Metaspace: 104K(320K)->104K(320K) NonClass: 98K(192K)-
>98K(192K) Class: 6K(128K)->6K(128K)
[2026-10-23T22:44:41.376-0400][0.017s][info][gc           ]
GC(0) Pause Young (Normal) (G1 Evacuation Pause) 9M->1M(18M)
0.293ms
```

Eden decreased from 8 regions to 0 showing that its allocated objects were processed. The survivor count increased from 0 to 1 showing that objects survived and moved into the survivor space. The old region count was still 2 so there was no net change in the number of old regions during collection. Total heap usage fell from 9MB to 1MB.

Q12. Phase 4 calls `.clear()` on both lists, removing their references to the retained objects. Does this guarantee that those objects will be garbage-collected before the program exits? Explain the difference between becoming eligible for garbage collection and having memory reclaimed. Refer to your log where relevant, but you do not need to identify the exact time Phase 4 occurred.

No, this does not guarantee that the objects will be garbage-collected before the program exits. Since calling .clear() on both of the lists removes the references of the lists to their retained objects, those objects become eligible for garbage collection. Just because they are eligible for collection does not mean that their memory is automatically reclaimed since the JVM decides when to run garbage collection, therefore, the program may terminate before garbage collection reclaims those objects.

Q13. In 2–3 sentences, explain the generational hypothesis and why it makes GC efficient. Use at least one concrete number from your `gc.log` to support your answer.

The generational hypothesis states that most objects die young, while objects that survive multiple garbage collections are more likely to live longer. This makes GC efficient because the JVM can be directed toward the young generation than scanning the full heap. In GC(0), the heap usage had dropped from 9MB to 1MB, while Eden decreased from 8 regions to 0.

Submission

Submit a single PDF named **A08_<BU-ID>.pdf**, replacing <BU-ID> with your BU ID.

Your PDF must include answers to **all questions, Q1–Q13**, including the annotated bytecode for Q7 and the requested output or log lines for Q8–Q10.

You may enter your answers directly in this document or use a separate document with each answer clearly labeled by question number.

When finished, save or export your document as a PDF:

- Microsoft Word on Windows: Select File → Save As, then choose PDF as the file type.
- Microsoft Word on Mac: Select File → Save As, then choose PDF from the File Format menu.
- Pages on Mac: Select File → Export To → PDF.
- Google Docs: Select File → Download → PDF Document (.pdf).

Open the PDF to check that all answers and code annotations are readable, then submit it through the Gradescope assignment link on Blackboard.